

SentinelAI

1. What problem are you solving?

Problem statement:

Digital payment fraud is often identified only after a suspicious transaction has already been processed. SentinelAI aims to detect high risk transactions before authorization by evaluating the transaction along with relevant behavioral and contextual signals.

Who is affected?

Customers, banks, fintech platforms and payment service providers.

Why does it matter?

Because once fraudulent money has moved, recovery becomes significantly harder. Detecting risk before authorization creates an opportunity to stop or verify the transaction.

2. Why is this problem important?

Digital payments are becoming increasingly common, while fraudsters are also becoming more sophisticated.

Traditional fraud detection can rely heavily on predefined rules and individual transaction attributes.

The gap SentinelAI addresses is **context**.

A transaction may not look suspicious by itself, but it can become suspicious when combined with things such as:

- New device
- New location
- New beneficiary
- Unusual amount
- Unusual transaction time

- Rapid transaction activity
- Suspicious receiver
- Deviation from normal user behavior

The important problem is therefore:

A transaction should not always be judged in isolation. Its surrounding context matters.

3. What is your solution?

SentinelAI is a real time pre authorization fraud detection system. It evaluates transaction and contextual signals, generates a risk assessment, and produces an actionable decision such as allowing, verifying, or blocking the transaction before authorization.

Core flow

Transaction initiated

↓

Transaction and contextual data

↓

Feature extraction / risk analysis

↓

Risk score

↓

Decision

ALLOW / VERIFY / BLOCK

That is the core. Don't add seventeen boxes just because PowerPoint allows rectangles.

4. What is innovative about your solution?

Our innovation is the contextual approach to fraud detection. Instead of evaluating only the transaction itself, SentinelAI considers multiple signals around the transaction and combines them into a single risk assessment before authorization.

The important word is **contextual**.

Not "we use AI."

Not "we have a dashboard."

Not "we use a database."

Those are implementation details.

5. How is it different from existing solutions?

Your USP:

SentinelAI does not simply ask, "Is this transaction unusual?" It asks, "Is this transaction unusual for this user in this context?"

Traditional systems may look at isolated rules or transaction characteristics.

SentinelAI combines multiple contextual signals to produce a risk based decision.

Your differentiation

Traditional

Transaction → Rule → Flag

SentinelAI

Transaction + Context + Behavior → Risk assessment → Action

And because your system is intended to operate **before authorization**, the output is actionable rather than merely being an after the fact fraud alert.

6. What technologies did you use and why?

Here we stick to **your actual stack**.

Do not say Kafka.

Do not say Azure unless you actually deploy there.

Do not invent technologies because they sound impressive.

Your answer should be structured around the components you actually built:

Frontend

Used for transaction interaction and displaying the fraud detection result and monitoring information.

Backend API

Handles transaction requests and connects the transaction flow with the fraud detection logic.

Database

Stores the structured information required by the system, including transaction and related contextual data according to your schemas.

AI/ML component

Used for the risk assessment component of SentinelAI.

Docker

Used to containerize the application and provide a consistent environment for running the system.

The exact technology names should match your **technical contract**, not some generic fintech architecture I made up five minutes ago.

7. How does each technology help?

Keep the answer functional.

The frontend provides the user and monitoring interface. The backend exposes the APIs and coordinates the transaction flow. The database provides the historical and structured data required for analysis. The AI/ML component evaluates transaction risk. Docker packages the application so it can be run consistently across environments.

Notice what is missing:

Kafka.

Azure.

Kubernetes.

Twenty seven cloud services nobody actually configured.

Good.

8. What is the impact and feasibility?

Impact

SentinelAI aims to:

Reduce fraudulent transactions

By identifying high risk activity before authorization.

Reduce unnecessary manual investigation

By assigning risk and providing decision information.

Improve customer protection

By detecting suspicious activity earlier.

Improve trust in digital payments

Because prevention is stronger than discovering fraud after the money has disappeared into the great financial void.

Feasibility

The prototype uses technologies that can already be deployed in real software systems.

The architecture is modular, so individual components can be improved without rebuilding the entire system.

9. How did you test the solution?

This needs to match the scenarios we've already discussed.

Normal transaction

Normal device, normal location, normal amount and normal behavior.

Expected: Low risk → Allow.

Unusual transaction

Transaction amount or timing deviates from the user's normal behavior.

Expected: Increased risk.

Account takeover

New device + new location + unusual activity.

Expected: High risk → Verify or Block.

New beneficiary

A new beneficiary combined with unusual transaction characteristics.

Expected: Elevated risk.

Rapid activity

Multiple transactions within a short period.

Expected: Increasing risk.

Suspicious receiver

Transaction involving a receiver with suspicious characteristics.

Expected: Higher risk.

Your key testing statement:

We tested the system using both normal and suspicious transaction scenarios to verify whether the risk assessment and resulting decision changed appropriately based on the contextual signals.

10. How will you scale your solution?

Only talk about things consistent with what you've built.

Our current prototype is containerized using Docker, which makes deployment consistent across environments. As transaction volume increases, the backend

can be scaled by running multiple instances, while the database can be optimized and scaled independently. The individual components can also be improved or replaced without changing the complete transaction flow.

If they ask:

"Are you using Kafka?"

No. Kafka is not part of our current prototype. Our current architecture uses the API based transaction flow. If transaction volume eventually required event streaming, that could be considered as a future architectural enhancement.

That's honest.

Honesty is surprisingly powerful in judging. Humans seem to appreciate it despite spending most of civilization trying to optimize around it.

11. What are the limitations?

Three strong ones:

Dataset limitation

The effectiveness of the risk model depends on the quality and diversity of the data available.

Prototype limitation

The hackathon version has not been tested at full production transaction volumes.

False positives

A highly sensitive fraud detection system could incorrectly flag legitimate transactions.

Then:

A production system would therefore require continuous evaluation, threshold tuning and feedback from confirmed fraud and legitimate transactions.

12. What will you do with the prize money?

We would use the prize money to take SentinelAI from a hackathon prototype toward a stronger real world proof of concept.

Priorities:

1. Improve datasets and model evaluation.
 2. Improve deployment and infrastructure.
 3. Perform stronger security and performance testing.
 4. Improve the product and monitoring interface.
-

13. Explain how you will use the funds

If they demand specifics:

We would prioritize improving the data and fraud detection model, followed by infrastructure and deployment, security and performance testing, and finally product improvements.

Don't invent exact percentages unless your team has actually agreed on them.

14. Explain your prototype

This is where you **show**, not lecture.

Your demo:

Normal transaction

Show a legitimate transaction.

Risk: Low

Decision: Allow

Then introduce suspicious signals.

New device

↓

New location

↓

New beneficiary

↓

Unusual amount

↓

Rapid activity

Then show the risk assessment changing.

Finally:

High Risk

Decision: Verify / Block

And show **why**.

That demonstrates the entire SentinelAI concept in a few seconds.

15. What is the potential for real world deployment?

This is your final architecture story:

Payment initiated

↓

SentinelAI API

↓

Transaction + contextual information

↓

Risk analysis

↓

Risk score



Decision engine



ALLOW / VERIFY / BLOCK



Authorization continues or is stopped

For deployment:

Our current prototype can be containerized and deployed using Docker. In a real financial environment, the same architecture could be integrated with an existing payment authorization system through APIs, with stronger infrastructure, security, monitoring, data pipelines and compliance controls added for production.